

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

MAESTRÍA EN SEGURIDAD INFORMÁTICA

TAREA 4 - PRÁCTICA EN APLICACIONES WEB CON OWASP

Explotación y mitigación de 10 vulnerabilidades OWASP en el laboratorio bWAPP

Autor (es):

Gerbert David García Loaiza - ggarcial17@miumg.edu.gt

Carné: 1293-21-11608

Profesor

Ing. Mario Roberto López Sajquin

Curso

SEGURIDAD EN APLICACIONES - 42026-1293-013-E

Guatemala, octubre de 2026

Tabla de Contenido

Introducción.....1
1. SQL Injection (GET/Search).....2
2. HTML Injection - Reflected (GET).....2
3. Cross-Site Scripting - Reflected (GET).....3
4. OS Command Injection.....4
5. Broken Authentication - Insecure Login Forms.....5
6. Insecure Direct Object Reference (IDOR) en el cambio de secreto.....6
7. XML External Entity (XXE).....7
8. Unrestricted File Upload.....8
9. Cross-Site Request Forgery (CSRF) en el cambio de contraseña.....9
10. Server Side Request Forgery (SSRF).....10
Conclusión.....12
Referencias.....13

Introducción

Este informe documenta la práctica de la Tarea 4 del curso Seguridad en Aplicaciones, que consistió en instalar el laboratorio bWAPP (bee-box v1.6) en una máquina virtual aislada y explotar, de forma controlada, diez vulnerabilidades distintas basadas en la clasificación OWASP. Para cada una se muestra una prueba exitosa documentada con una captura de pantalla, se describe la amenaza, se asigna un nivel de riesgo y se propone una mitigación.

bWAPP es una aplicación web creada a propósito con fallas de seguridad, diseñada para el aprendizaje de pruebas de penetración. Todas las pruebas de este informe se realizaron contra una instancia local de bWAPP (bee-box v1.6 sobre Ubuntu, en VirtualBox), en una red NAT aislada sin salida a Internet ni a la red del equipo anfitrión, y únicamente contra el puerto reenviado hacia esa máquina virtual (127.0.0.1:8081). No se accedió a ningún sistema ajeno al laboratorio.

1. SQL Injection (GET/Search)

Amenaza OWASP: A1 - Injection (SQL Injection)

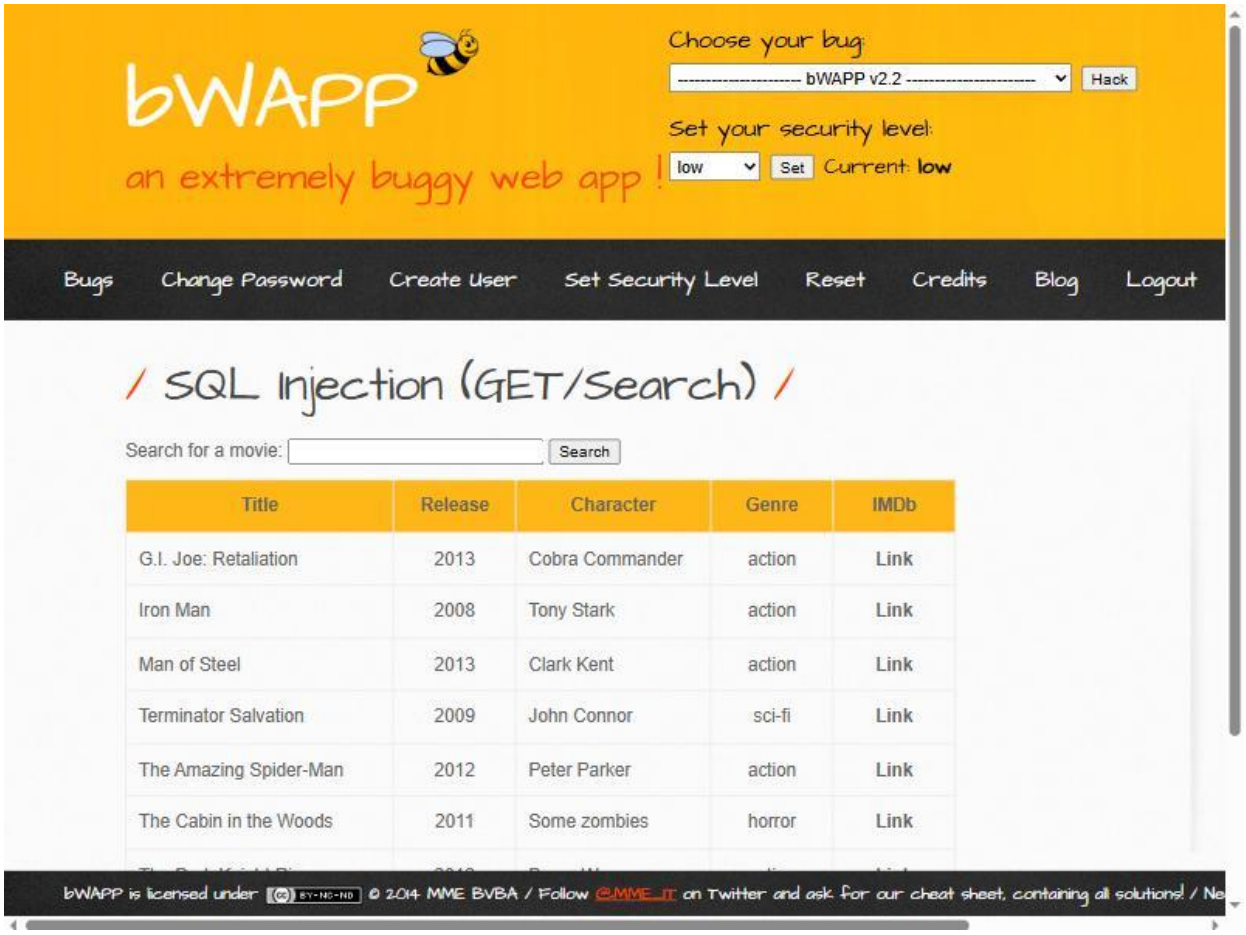
Descripción de la amenaza. El parámetro title del buscador de películas se concatena directamente dentro de una consulta SQL, sin validarse ni parametrizarse. Al enviar el valor ' OR '1'='1, la condición WHERE se vuelve siempre verdadera y la consulta devuelve toda la tabla en lugar de respetar el filtro de búsqueda. El problema ocurre porque la aplicación arma la sentencia SQL mediante la concatenación de cadenas de texto, en vez de usar sentencias preparadas.

Nivel de riesgo: Alto

Mitigación. Usar consultas parametrizadas o sentencias preparadas (por ejemplo, PDO o mysqli con bind_param), y nunca concatenar la entrada del usuario dentro del SQL. Se recomienda además validar el tipo y la longitud del parámetro, y aplicar el principio de mínimo privilegio en la cuenta de base de datos, para que no tenga permisos de lectura sobre tablas que no necesita.

Figura 1

Evidencia de la explotación · SQL Injection (GET/Search)



2. HTML Injection - Reflected (GET)

Amenaza OWASP: A1 - Injection (HTML Injection)

Descripción de la amenaza. El formulario de nombre y apellido refleja el parámetro firstname directamente en el HTML de la respuesta, sin escapar los caracteres especiales < y >. Esto permite inyectar etiquetas HTML arbitrarias que el navegador interpreta como marcado real, lo que puede alterar la apariencia de la página o preparar el terreno para ataques más serios, como el phishing o la desfiguración del sitio.

Nivel de riesgo: Medio

Mitigación. Escapar toda salida que incluya datos del usuario con una función como htmlspecialchars() en PHP antes de imprimirla en el HTML, o usar un motor de plantillas que escape por defecto. Como defensa adicional, se recomienda aplicar una política de Content-Security-Policy.

Figura 2

Evidencia de la explotación · HTML Injection - Reflected (GET)



3. Cross-Site Scripting - Reflected (GET)

Amenaza OWASP: A3 - Cross-Site Scripting (XSS)

Descripción de la amenaza. El mismo parámetro firstname no filtra etiquetas <script>, lo que permite ejecutar código JavaScript arbitrario en el navegador de la víctima. Un atacante podría robar la cookie de sesión, redirigir al usuario a otro sitio o realizar acciones en su nombre. El problema ocurre porque la aplicación confía en la entrada del usuario y la refleja sin sanitizarla ni escaparla.

Nivel de riesgo: Alto

Mitigación. Escapar y codificar toda entrada que se refleje en el HTML, usar una cabecera Content-Security-Policy restrictiva y marcar las cookies de sesión como HttpOnly y Secure, para que no puedan leerse desde JavaScript.

Figura 3

Evidencia de la explotación · Cross-Site Scripting - Reflected (GET)



4. OS Command Injection

Amenaza OWASP: A1 - Injection (Command Injection)

Descripción de la amenaza. El módulo de búsqueda DNS pasa el valor del campo target directamente a una llamada del sistema operativo (de tipo nslookup o dig), sin sanitizar los separadores de comandos como &&, ; o |. Esto permite encadenar comandos arbitrarios del sistema con los mismos privilegios del servidor web.

Nivel de riesgo: Alto

Mitigación. Nunca pasar entrada del usuario a funciones que invocan directamente al shell, como exec, system o shell_exec. Cuando sea indispensable, usar listas blancas estrictas de caracteres permitidos y funciones que llamen al binario con argumentos separados, sin pasar por el shell, combinadas con la validación del formato, por ejemplo que el valor sea una IP o un dominio válido.

Figura 4

Evidencia de la explotación · OS Command Injection



5. Broken Authentication - Insecure Login Forms

Amenaza OWASP: A2 - Broken Authentication and Session Management

Descripción de la amenaza. El formulario de inicio de sesión se sirve por HTTP sin cifrado y además muestra en el propio formulario las credenciales válidas de ejemplo. Al no cifrarse el tráfico, cualquier persona en la misma red puede capturar el usuario y la contraseña con un analizador de tráfico, y mostrar credenciales de ejemplo en pantalla facilita el reconocimiento para un atacante.

Nivel de riesgo: Alto

Mitigación. Forzar el uso de HTTPS en todo el flujo de autenticación, incluida la cabecera HSTS; nunca mostrar credenciales reales ni de ejemplo en los campos del formulario en producción; aplicar un límite de intentos de inicio de sesión con bloqueo temporal, y registrar los intentos fallidos.

Figura 5

Evidencia de la explotación · Broken Authentication - Insecure Login Forms



6. Insecure Direct Object Reference (IDOR) en el cambio de secreto

Amenaza OWASP: A4 - Insecure Direct Object References

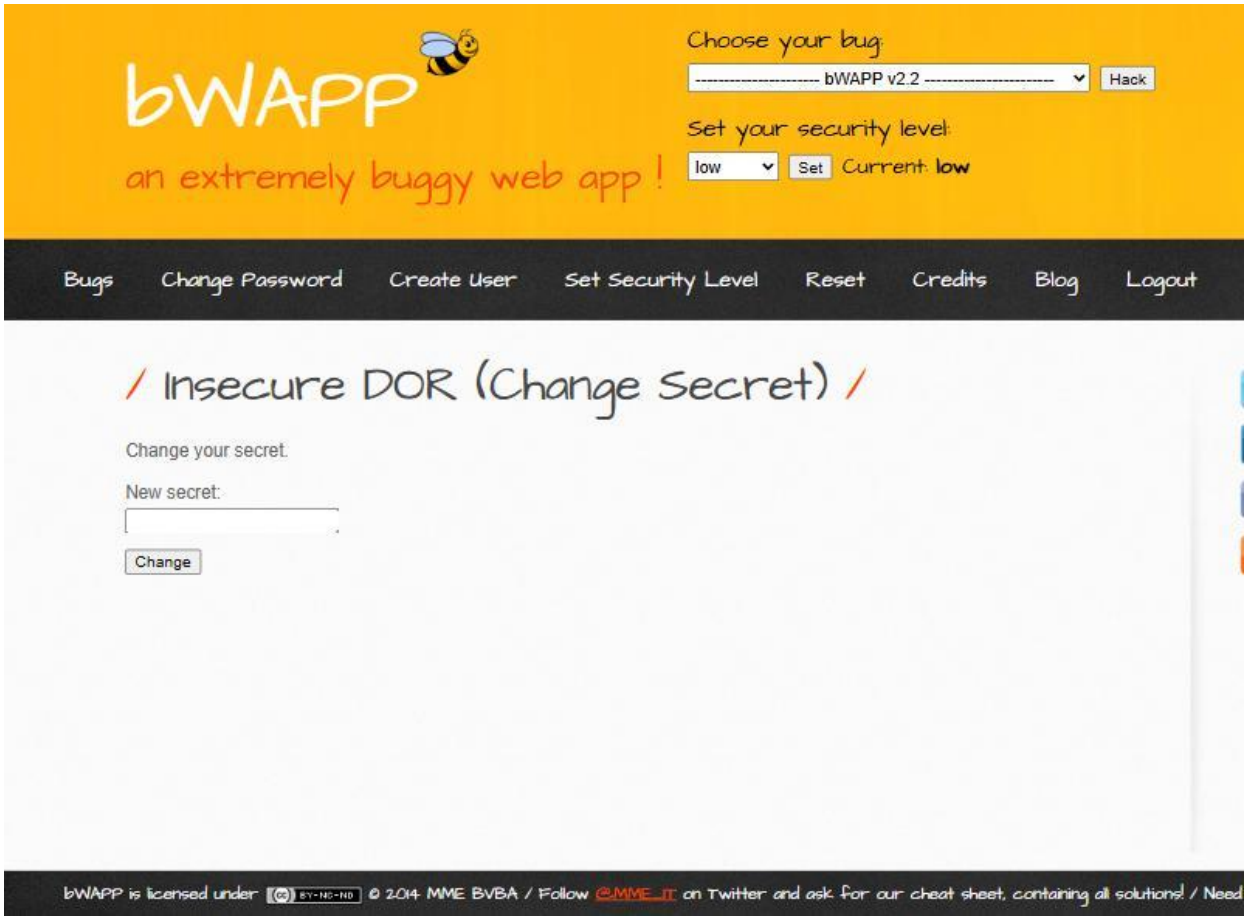
Descripción de la amenaza. El formulario para cambiar el secreto de la cuenta envía un campo oculto login con el nombre del usuario propietario del registro. El servidor no verifica que el usuario autenticado en la sesión coincida con el valor de ese campo antes de aplicar el cambio, por lo que, si se modifica el campo login en la petición, se puede sobrescribir el secreto de cualquier otra cuenta sin autorización.

Nivel de riesgo: Alto

Mitigación. No confiar en identificadores de objeto enviados por el cliente, como campos ocultos o parámetros, para tomar decisiones de autorización. El servidor debe obtener el usuario objetivo únicamente de la sesión autenticada y validar explícitamente que el recurso solicitado pertenece al usuario actual antes de modificarlo.

Figura 6

Evidencia de la explotación · Insecure Direct Object Reference (IDOR) en el cambio de secreto



7. XML External Entity (XXE)

Amenaza OWASP: A4/A7 - XML External Entity Attacks

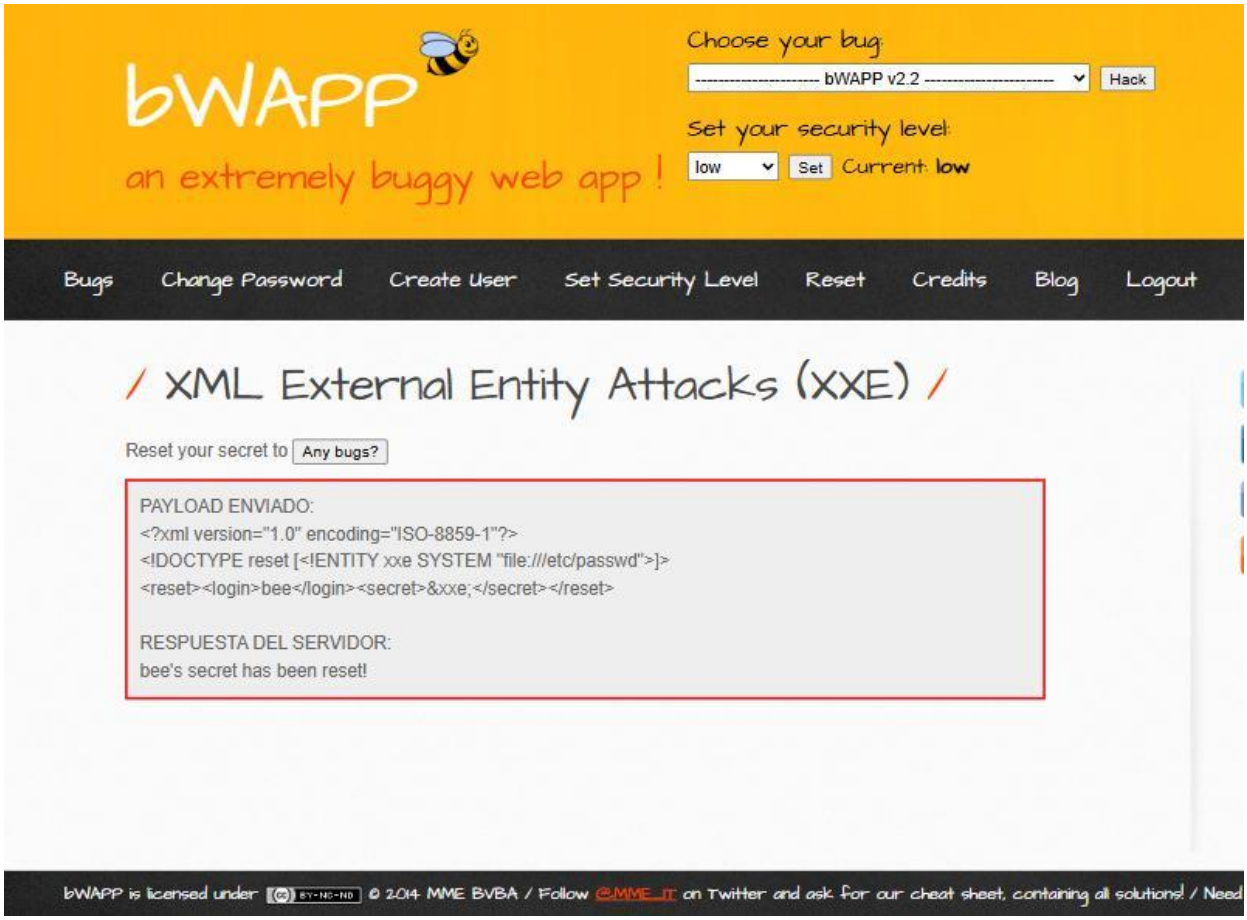
Descripción de la amenaza. El endpoint que restablece el secreto del usuario acepta un cuerpo en formato XML y lo procesa con un analizador que no deshabilita la resolución de entidades externas. Esto permite declarar una entidad de tipo SYSTEM que apunta a un archivo local del servidor, con lo que el analizador la resuelve y expone su contenido. El mismo mecanismo podría usarse para leer otros archivos, hacer peticiones internas o provocar una denegación de servicio.

Nivel de riesgo: Alto

Mitigación. Deshabilitar la carga de entidades externas y de tipos de documento (DTD) en el analizador XML, validar el XML recibido contra un esquema estricto, y preferir un formato menos permisivo, como JSON, cuando no se necesite XML.

Figura 7

Evidencia de la explotación · XML External Entity (XXE)



8. Unrestricted File Upload

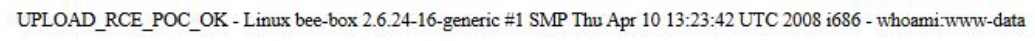
Amenaza OWASP: A5 - Security Misconfiguration

Descripción de la amenaza. El formulario para subir imágenes no valida el tipo real del archivo, ni por extensión, ni por tipo de contenido, ni por su contenido real, antes de guardarlo en una carpeta que el servidor expone públicamente. Esto permite subir un archivo con código ejecutable y luego invocarlo directamente desde el navegador, con lo que se ejecuta código de forma remota en el servidor.

Nivel de riesgo: Alto

Mitigación. Validar la extensión y el contenido real del archivo mediante sus primeros bytes, renombrar los archivos subidos con nombres aleatorios y sin extensión ejecutable, guardarlos fuera de la carpeta pública del sitio o en un almacenamiento que no ejecute scripts, y servirlos siempre con la cabecera Content-Disposition en modo adjunto cuando corresponda.

Figura 8
Evidencia de la explotación · Unrestricted File Upload



9. Cross-Site Request Forgery (CSRF) en el cambio de contraseña

Amenaza OWASP: A8 - Cross-Site Request Forgery

Descripción de la amenaza. El endpoint que cambia la contraseña acepta la petición por el método GET, sin ningún token contra CSRF, sin verificar el origen de la petición y sin pedir la contraseña actual. Como el navegador adjunta automáticamente la cookie de sesión en cualquier petición al sitio, basta con que la víctima, ya autenticada, visite una página externa controlada por un atacante para que su contraseña cambie sin su conocimiento ni su consentimiento.

Nivel de riesgo: Alto

Mitigación. Usar un token único contra CSRF por sesión o por petición y validarlo en el servidor; exigir que las operaciones sensibles, como el cambio de contraseña, se hagan por el método POST y requieran la contraseña actual; y configurar las cookies de sesión con el atributo SameSite en modo Strict o Lax.

Figura 9

Evidencia de la explotación · Cross-Site Request Forgery (CSRF) en el cambio de contraseña



10. Server Side Request Forgery (SSRF)

Amenaza OWASP: A10 - Server-Side Request Forgery

Descripción de la amenaza. El módulo de selección de idioma incluye como código PHP el archivo indicado en un parámetro, sin validarlo contra una lista cerrada de valores permitidos. Combinado con un script que abre conexiones hacia direcciones y puertos arbitrarios, esto permite forzar al servidor a conectarse hacia su propia red interna, y así revela qué servicios internos están disponibles sin que el atacante tenga acceso directo a esa red.

Nivel de riesgo: Alto

Mitigación. No incluir ni ejecutar archivos a partir de un parámetro que controla el usuario; usar una lista blanca estricta de nombres de archivo permitidos, sin rutas. Para cualquier función que haga peticiones salientes a una dirección proporcionada por el usuario, validar y restringir los destinos permitidos, bloquear los rangos de red privados y locales, y ejecutar esas peticiones desde una red segmentada sin acceso a servicios internos sensibles.

Figura 10

Evidencia de la explotación · Server Side Request Forgery (SSRF)



Conclusión

La práctica confirmó que diez problemas distintos de seguridad, cubiertos por las categorías OWASP de inyección, autenticación, control de acceso, exposición de datos, configuración y falsificación de peticiones, pueden explotarse con pasos relativamente simples cuando la aplicación no valida la entrada del usuario, no separa los privilegios y no protege sus formularios contra el origen de la petición. En todos los casos la causa raíz es la misma: la aplicación confía en información que el cliente puede manipular, ya sea un parámetro de búsqueda, un campo oculto, un archivo subido o el cuerpo de una petición.

Las mitigaciones propuestas para cada hallazgo comparten principios comunes: validar y sanear toda entrada del usuario en el servidor, nunca en el cliente; usar mecanismos del lenguaje y del framework que eviten el error por diseño, como las sentencias preparadas o el escape automático de salida; aplicar el principio de mínimo privilegio tanto en la base de datos como en el sistema operativo; y verificar siempre, del lado del servidor, que el usuario autenticado tiene permiso sobre el recurso que solicita. Aplicar estos principios de forma sistemática, y no solo parchear cada síntoma por separado, es lo que reduce de verdad la superficie de ataque de una aplicación web.

Referencias

Open Worldwide Application Security Project. (s. f.). *OWASP Top 10*. https://owasp.org/www-project-top-ten/

MME BVBA. (2014). *bWAPP: A buggy web application* [Software]. SourceForge. https://sourceforge.net/projects/bwapp/